

Step 3 — A Triage Agent That Extracts a Structured Ticket From Free Text — Solution

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Approach 1 — extraction plus three of the four failures

```
# schemas.py
from typing import Literal
from pydantic import BaseModel

class SupportTicket(BaseModel):
    customer_name: str | None
    issue_category: Literal["billing", "technical", "account_access", "other"]
    urgency: Literal["low", "medium", "high"]
    summary: str
```

```
# test_data.py
TEST_INPUTS = [
    "Hi, I'm Sarah Khan. I was charged twice for my subscription this month and I",
    "can't log into my account, tried resetting password twice, still nothing",
    "just wondering what your refund policy is, no rush",
]
```

```
# chat_client.py (added)
from openai import AuthenticationError, RateLimitError
from pydantic import ValidationError
from schemas import SupportTicket

def triage_extract(client, message):
    response = client.beta.chat.completions.parse(
        model="gpt-4o-mini",
        messages=[
            {"role": "system", "content": "Extract a support ticket. Leave custom"},
            {"role": "user", "content": message},
```

```
    ],  
    response_format=SupportTicket,  
)  
    return response.choices[0].message.parsed
```

```
# main.py, /extract mode  
try:  
    ticket = triage_extract(client, user_text)  
    print(ticket)  
except AuthenticationError:  
    print("Your API key stopped working.")  
except RateLimitError:  
    print("Rate limited – try again in a moment.")  
except ValidationError:  
    print("Couldn't build a clean ticket from that message.")
```

This covers three of the four failures and passes the 3 test inputs. It's missing the startup key check and the too-long-conversation case, doesn't log anything (just prints), and gives up immediately on any validation failure without trying again.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — all four failures, startup check, and a real test file

```
# schemas.py
from typing import Literal
from pydantic import BaseModel

class SupportTicket(BaseModel):
    customer_name: str | None
    issue_category: Literal["billing", "technical", "account_access", "other"]
    urgency: Literal["low", "medium", "high"]
    summary: str
```

```
# chat_client.py (added)
from openai import OpenAI, AuthenticationError, RateLimitError, BadRequestError
from pydantic import ValidationError
from schemas import SupportTicket

def check_api_key_at_startup(client: OpenAI) -> None:
    try:
        client.chat.completions.create(
            model="gpt-4o-mini",
            messages=[{"role": "user", "content": "ping"}],
            max_tokens=1,
        )
    except AuthenticationError:
        raise SystemExit("Your OPENAI_API_KEY is invalid. Fix your .env before co

def triage_extract(client: OpenAI, message: str) -> SupportTicket:
    response = client.beta.chat.completions.parse(
        model="gpt-4o-mini",
        messages=[
            {
                "role": "system",
                "content": "Extract a support ticket from the customer's message.
                            Leave customer_name empty if it isn't mentioned – ne
```

```

        },
        {"role": "user", "content": message},
    ],
    response_format=SupportTicket,
)
return response.choices[0].message.parsed

```

```

# main.py, /extract mode
try:
    ticket = triage_extract(client, user_text)
    print(ticket)
except AuthenticationError:
    logger.error("API key stopped working mid-session")
    print("Something's wrong with the connection – try again shortly.")
except RateLimitError:
    logger.warning("Rate limited during extraction")
    print("We're being rate-limited – wait a moment and try again.")
except BadRequestError as e:
    # covers the too-long-conversation case for many SDK versions
    logger.warning("Request rejected, likely too long: %s", e)
    print("That conversation got too long for one request – try starting a new one")
except ValidationError as e:
    logger.warning("Model output didn't match SupportTicket: %s", e)
    print("Couldn't extract a clean ticket from that – try rephrasing.")

```

```

# test_data.py
TEST_INPUTS: list[str] = [
    "Hi, I'm Sarah Khan. I was charged twice for my subscription this month and I",
    "can't log into my account, tried resetting password twice, still nothing",
    "just wondering what your refund policy is, no rush",
]

# test_chat_client.py – a quick manual check, not a full test suite yet
from test_data import TEST_INPUTS
from chat_client import create_client, triage_extract
from config import load_config

config = load_config()
client = create_client(config)

for text in TEST_INPUTS:
    ticket = triage_extract(client, text)
    print(text, "->", ticket)
    assert ticket.customer_name is None or isinstance(ticket.customer_name, str)

```

```
# specifically confirm input 2's no-name case
second = triage_extract(client, TEST_INPUTS[1])
assert second.customer_name is None, "expected no name to be guessed for input 2"
```

Difference from Basic: `check_api_key_at_startup()` catches a bad key before the user ever gets to type a message, exactly like the README's Problem/Fix table asks for. Each failure has its own log line at the right level (`error` for a broken key, `warning` for things that are more like normal operating conditions) and its own user-facing message, so a person using the app can tell *what kind* of problem happened. `test_chat_client.py` explicitly asserts on the input-2 no-name case instead of just eyeballing the printed output. This version still only ever tries once — any validation failure is final, even a borderline one that a clearer instruction might have fixed.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — retry once with a clarifying instruction

```
def triage_extract(client: OpenAI, message: str, extra_instruction: str = "") ->
    system_content = "Extract a support ticket. Leave customer_name empty if not"
    if extra_instruction:
        system_content += " " + extra_instruction
    response = client.beta.chat.completions.parse(
        model="gpt-4o-mini",
        messages=[
            {"role": "system", "content": system_content},
            {"role": "user", "content": message},
        ],
        response_format=SupportTicket,
    )
    return response.choices[0].message.parsed

def triage_extract_with_retry(client: OpenAI, message: str) -> SupportTicket:
    try:
        return triage_extract(client, message)
    except ValidationError as first_error:
        logger.warning("First extraction attempt failed validation: %s", first_error)
        clarification = (
            "Your previous answer didn't match the required ticket format. "
            "issue_category must be exactly one of: billing, technical, account_a
            "urgency must be exactly one of: low, medium, high. Try again, carefu
        )
        try:
            return triage_extract(client, message, extra_instruction=clarification)
        except ValidationError as second_error:
            logger.error("Second extraction attempt also failed validation: %s", second_error)
            raise
```

Expected behavior: on the 3 normal test inputs, the retry path never triggers —

`triage_extract()` succeeds on the first try, exactly like Intermediate. Force the retry by testing with a deliberately awkward message and watch the log: one `WARNING` from the first failure, then either a successful return or an `ERROR` from a second failure that gets re-raised.

Approach 2 — tailor the retry to *why* the first attempt likely failed

```
def triage_extract_with_retry(client: OpenAI, message: str) -> SupportTicket:
    try:
        return triage_extract(client, message)
    except ValidationError as first_error:
        logger.warning("First extraction attempt failed validation: %s", first_error)
        if len(message.split()) < 4:
            # likely zero-info: too little was said to classify confidently
            clarification = (
                "The customer's message is very short. If you genuinely cannot tell me
                'the issue_category, use 'other' and keep urgency as 'low'."
            )
        else:
            # likely a formatting/judgment miss, not a lack of information
            clarification = (
                "Your previous answer didn't match the required format. "
                "issue_category must be exactly one of: billing, technical, account,
                urgency must be exactly one of: low, medium, high. Re-read the message."
            )
        try:
            return triage_extract(client, message, extra_instruction=clarification)
        except ValidationError as second_error:
            logger.error("Second extraction attempt also failed validation: %s", second_error)
            raise
```

The word-count check here is deliberately rough. A short message can still be a real, confident judgment call, and a long one can still lack real information. This is a starting guess at which clarification will help most — not a solved answer for "zero info" versus "got it wrong."

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate treats every validation failure the same way — catch it, tell the user, stop. Approach 1 gives the model one more try with a clearer instruction, no matter why the first attempt failed. Approach 2 makes a rough guess at *why* it failed — too little information, or a formatting slip — and picks the retry instruction to match. That's how the extra API call earns its cost: it's smarter than just asking again the same way.

Which one should you actually write? Approach 1, for most real use. It's simpler, and one clear, specific clarifying instruction usually fixes a formatting slip no matter the underlying reason. Reach for Approach 2's zero-info-vs-wrong distinction only once you're looking at real logs and see a specific, repeated pattern of failures that a generic clarification isn't fixing. Match the fix to the pattern you actually see, not the one you imagine in advance.